ARSENY NN · AUTOMATION & AI SYSTEMS

Строю системы автоматизации для бизнеса

CRM-процессы, Telegram-боты, LLM-обработка текста, автогенерация документов, парсеры, API-интеграции и операционная аналитика.

Арсений Николаевич Р. / @arsenynn

Telegram: @arsenynnn

Email: arsenynickolaevich@yandex.ru

portfolio.arsenynn.com/it-ai

РЕЗУЛЬТАТЫ АВТОМАТИЗАЦИИ

Цифры, которые понятны бизнесу без чтения кода

КОММЕРЧЕСКИЕ ПРЕДЛОЖЕНИЯ

2–4 часа → 5 минут

Автогенерация КП из брифа, CRM-данных и шаблонов.

G20

ДОКУМЕНТЫ

1–2 дня → 5 минут

Договоры, счета и акты через Google Docs/PDF.

G20

ОТВЕТ НА ЛИД

до 10 минут

SLA-уведомления и единая обработка заявок.

G20

ЗАЯВКИ ЧЕРЕЗ БОТ/САЙТ

55%

Входящие заявки проходят через управляемый digital-flow.

G20

КОНТРОЛЬ ДУБЛЕЙ

100%

Каждая вакансия обрабатывается один раз по ID.

JOB AUTOMATION

ЧТО ДЕЛАЮ

Собираю рабочие системы, которые убирают ручную операционку

Автоматизация лидов, CRM, документов, отчётов, Telegram-ботов, LLM-обработки, парсеров, API/Webhook-интеграций и внутренних инструментов.

Подход: сначала бизнес-задача и измеримый результат, затем стек, архитектура и технические детали.

ПРОЕКТЫ

Кейсы автоматизации: от заявки и CRM до LLM-обработки и аналитики

01 · CRM + EVENT PRODUCTION

G20 — автоматизация техпродакшна мероприятий

Спроектировал и реализовал операционную систему для event-production на базе Google Workspace, Apps Script, Python и Telegram: CRM, калькулятор КП, генерация документов и счетов, календарное планирование, архив, клиентская база, интеграции Avito/SiteChat/EventRadar, Telegram-уведомления, мониторинг и AI-подбор оборудования.

ЗАДАЧА

Коммерческие предложения готовились вручную 2–4 часа, документы занимали 1–2 дня, часть данных копировалась вручную, а заявки могли теряться между сайтом, чатами и таблицами.

РЕШЕНИЕ

CRM-ядро на Google Sheets, Apps Script-автоматизация, Python Telegram-бот, генерация КП/счетов/документов, календарное планирование, архив, клиентская база, интеграции Avito/SiteChat/EventRadar, мониторинг и AI-подбор оборудования.

КАК РАБОТАЕТ

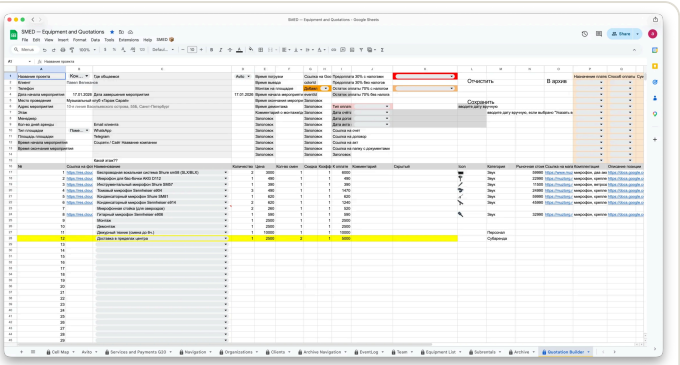
Заявка с сайта, Avito, бота или EventRadar попадает в Telegram/CRM → менеджер собирает смету в Quotation Builder → система считает стоимость, прибыль и расходы → генерирует КП/счёт/PDF → создаёт календарные события → отправляет уведомления → сохраняет архив и аналитику.

РОЛЬ

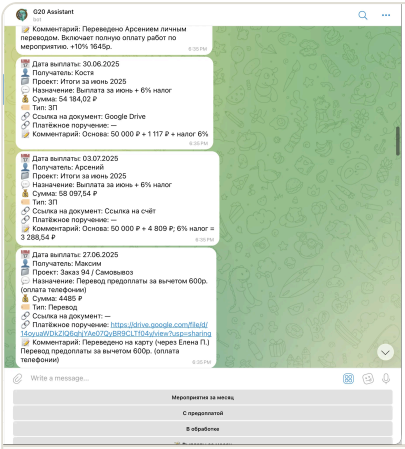
Архитектура системы, CRM, Telegram-бот, Apps Script-автоматизация, генерация документов, аналитика и интеграции.

РЕЗУЛЬТАТЫ

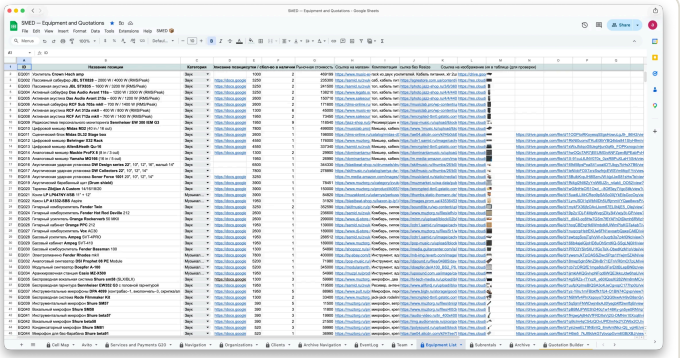
- Коммерческие предложения: 2–4 часа → 5 минут
- Документы: 1–2 дня → 5 минут
- Ответ на лид: до 10 минут
- Заявки через бот/сайт: 55%
- 20+ скриптов Apps Script для CRM, документов и уведомлений
- Меньше ошибок за счёт автозаполнения и валидаций



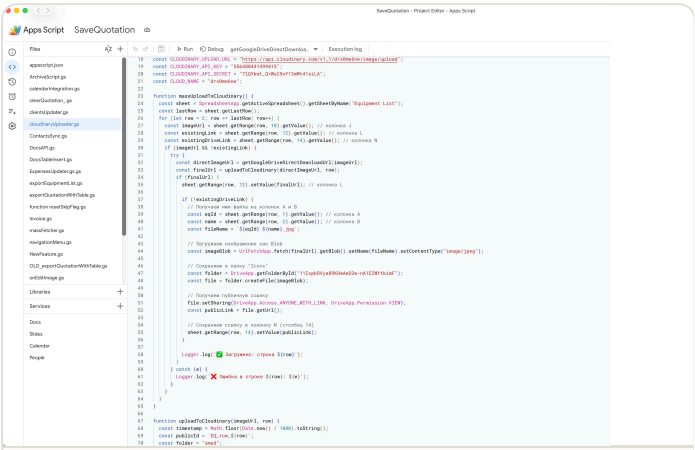
Конструктор КП: позиции, смены, коэффициенты, скидки и итоговая стоимость.



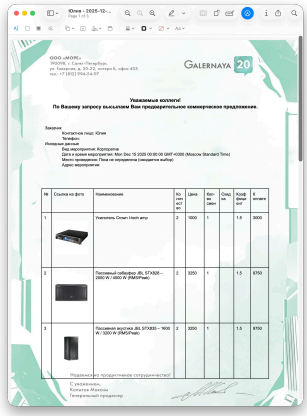
Telegram-бот для уведомлений, выплат, статусов проектов и операционного контроля.



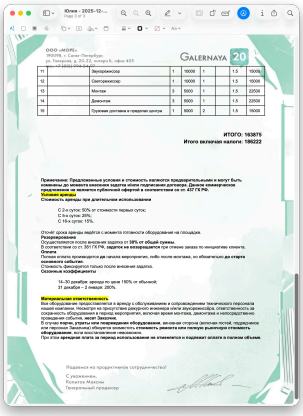
Справочник оборудования: категории, цены, рыночная стоимость, изображения и ссылки.



20+ Apps Script скриптов



PDF/Docs-коммерческое предложение с таблицей оборудования и автозаполненными данными.



Итоги, налоги, условия аренды и юридический блок в финальном документе.

ЧТО ВНУТРИ СИСТЕМЫ

Шесть модулей, которые превращают G20 в операционную систему

CRM & Project Core

- CRM-ядро на Google Sheets: проекты, клиенты, оборудование, навигация, архив, команда и расходы.
- Создание нового проекта из шаблона КП с автоматической нумерацией, формулами, стилями и служебными полями.
- Navigation-панель по активным проектам: статус, клиент, дата, площадка, суммы, прибыль и ссылка на проект.
- Архивирование проектов: перенос в Archive Navigation, скрытие листа и сохранение архивной копии.
- Синхронизация клиентов: client ID, поиск дублей по телефону, история проектов клиента.

Quotation Builder

- Конструктор коммерческих предложений в Google Sheets.
- Автозаполнение количества, смен, коэффициентов, скидок, стоимости и итогов.
- Расчёт по формуле: количество × цена × смены × коэффициент × скидка.
- Логика коэффициентов по количеству смен для оборудования и персонала.
- Equipment List: категории, цены, рыночная стоимость, описания, ссылки и изображения.
- Автодобавление новых позиций оборудования в справочник.

Finance, Documents & Payments

- Расчёт итогов, налогов, расходов, чистой прибыли, 25% и 50% от прибыли.
- Генерация коммерческого предложения в Google Docs по шаблону.
- Замена плейсхолдеров клиента, телефона, типа события, даты, площадки и адреса.
- Вставка таблицы оборудования и изображений в Google Docs через API.
- Добавление юридического блока из отдельного Google Docs-документа.
- Генерация счёта, PDF-экспорт и QR-код для оплаты.

Calendar, Archive & Operations

- Интеграция с Google Calendar: события создаются и обновляются из данных проекта.
- Два календаря: общий календарь событий и календарь подтверждённых/оплаченных проектов.
- Статус предоплаты управляет созданием события в Confirmed & Paid.
- Event ID сохраняется в таблицу, чтобы не создавать дубли.
- Telegram-отчёт по активным КП без предоплаты.
- Кастомное меню Google Sheets для быстрых действий и навигации.

Telegram Bot & Integrations

- Telegram-бот G20 Assistant на Python с модульным рантаймом g20bot.
- Интеграции Avito Messenger, site chat и EventRadar через webhook.
- Site chat валидирует payload, CORS и токен авторизации, затем отправляет лиды в Telegram.
- EventRadar-лиды валидируются, форматируются и отправляются в отдельный чат.
- Дедупликация лидов по chat/message ID.
- SQLite outbox: если Telegram/API недоступны, сообщения сохраняются и отправляются позже.

Reliability, AI & Monitoring

- Централизованная конфигурация через .env и JSON-логирование с маскированием секретов.
- Retry, timeout, rate limiting и circuit breaker для внешних API.
- Idempotency-хранилище, чтобы не обрабатывать Telegram update повторно.
- Prometheus /metrics, /healthz, heartbeat и Telegram-алерты администраторам.
- AI-помощник smed_assistant подбирает комплект оборудования по описанию мероприятия.
- Экспорт базы оборудования в JSON и отправка AI-подбора обратно в Google Sheets.

▼ Показать полный technical scope

- CRM-ядро на Google Sheets с листами для проектов, клиентов, оборудования, навигации, архива, команды и расходов.
- Финансовые расчёты: итог, налоги, расходы, чистая прибыль, 25% и 50% от чистой прибыли.
- Архивный экспорт КП в отдельный Google Sheets-файл и сохранение архивной копии на Google Drive.
- Генерация КП в Google Docs по шаблону с плейсхолдерами, таблицей оборудования, изображениями и юридическим блоком.
- Интеграция с Google Calendar: общий календарь событий и календарь Confirmed & Paid без дублей по Event ID.
- Конструктор КП с автозаполнением количества, смен, коэффициентов, скидок, стоимости и итогов.
- Создание проекта из шаблона с копированием диапазонов, формул, стилей, ширин колонок и служебных полей.
- Синхронизация клиентов, генерация client ID, поиск дублей по телефону и история проектов клиента.
- Генерация счёта, QR-код оплаты и PDF-экспорт.
- Кастомное меню Google Sheets, защита важных зон таблицы и очистка шаблонов без удаления формул.

- Google People API для подрядчиков, выпадающие списки и добавление новых подрядчиков в Google Contacts.
- Загрузка и обработка изображений оборудования: Pexels API, Drive, ресайз 300×300, Cloudinary и сохранение ссылок.
- Экспорт базы оборудования в JSON и AI-помощник smed_assistant на OpenAI API для подбора комплекта оборудования.
- Python-скрипты для создания листов проектов и отправки AI-подбора обратно в Quotation Builder.
- G20 Assistant на Python: модульный runtime, .env, JSON-логирование, retry, timeout, rate limiting и circuit breaker.
- SQLite outbox, idempotency-хранилище, нарезка длинных сообщений и fallback-доставка Telegram-уведомлений.
- Интеграции Avito Messenger, SiteChat и EventRadar через webhook с дедупликацией лидов.
- Prometheus /metrics, /healthz, heartbeat, latency, retries, circuit breaker state, outbox size и Telegram-алерты.
- Dockerfile и инструкции запуска через Docker, systemd и launchd.
- Легаси-бот: меню Telegram, Google Sheets, касса, выплаты, напоминания, Avito-опрос, дневные summary через Groq, OCR/ASR для аудио, PDF, DOCX и изображений.

Google Sheets | Apps Script | Python | Telegram Bot API | Google Docs API | Drive API | Calendar API | OpenAI API | Avito API | SQLite | Prometheus | REST/Webhooks/JSON

Portfolio repo: https://github.com/workflowarsenynn/event-crm-gworkspace-automation-portfolio

02 · PYTHON + AI

Автоматизация откликов на вакансии HeadHunter

Browser-first платформа на Python + Playwright для поиска вакансий, LLM-анализа JD, генерации персональных писем, заполнения анкет и controlled auto-submit с Telegram approval.

ЗАДАЧА

Ручной поиск, копирование описаний, выбор резюме и отклики занимали время, создавали дубли и плохо масштабировались. После ограничений API нужно было перенести автоматизацию в браузерный контур.

РЕШЕНИЕ

Система управляет Chromium через Playwright: ищет вакансии, извлекает описание, анализирует его через GPT, выбирает резюме, генерирует письмо, готовит отклик и отправляет его только через безопасный workflow. Простые вакансии отправляются автоматически, анкеты и нестандартные кейсы уходят в Telegram approval/handoff.

КАК РАБОТАЕТ

Search → fetch details → LLM classification → cover letter → prepare apply → safety check → auto-submit или Telegram approval → логирование → retry/status tracking.

РОЛЬ

Архитектура browser-first runtime, Playwright flow, LLM-промпты, safety layer, anti-duplicates, Telegram approval, retry queue, dashboard и regression smoke.

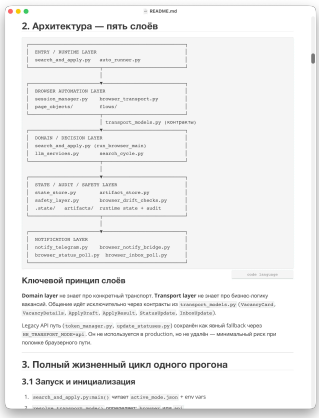
РЕЗУЛЬТАТЫ

- 100% контроль дублей по ID вакансии
- Controlled auto-submit для простых вакансий
- Telegram handoff для анкет и сложных кейсов
- Questionnaire engine для text, radio, checkbox, select
- Retry queue между сессиями и safety layer
- Trace artifacts, dashboard и regression smoke

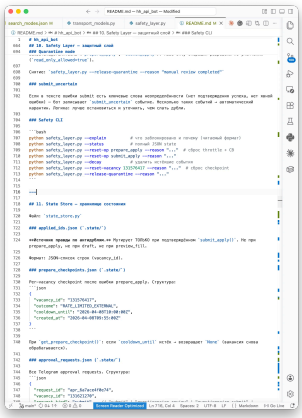
ПОДРОБНЫЙ РАЗБОР

Как и почему я строил browser-first автоотклики: архитектура, safety layer, handoff и ограничения.

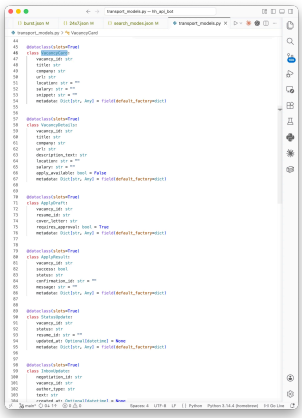
https://t.me/annsflow/136



Пятислойная архитектура: runtime, browser automation, decision layer, state/audit/safety и notification layer.

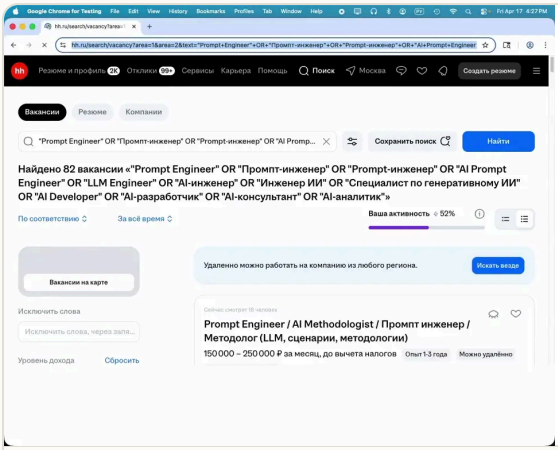


Safety Layer и CLI-восстановление: throttling, quarantine, submit_uncertain, reset операций и checkpoint recovery.

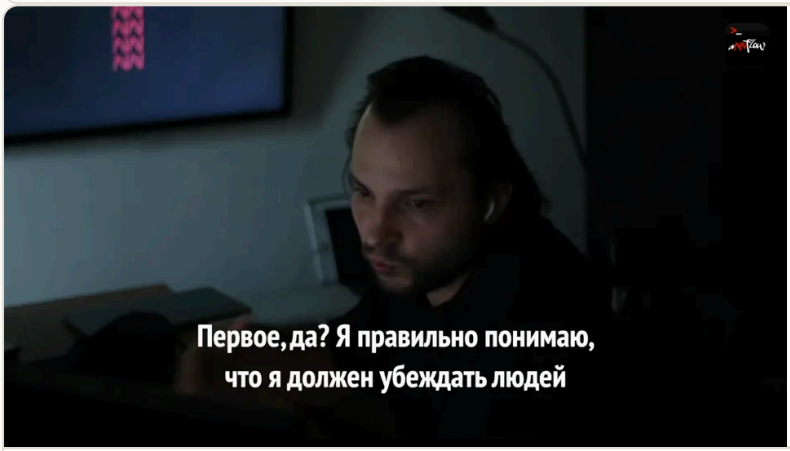


Типизированные transport contracts: единые dataclass-модели для вакансий, деталей, черновиков отклика, результатов submit, статусов и inbox-сообщений.

ВИДЕО PROOF



Демо browser-first процесса: система работает в браузере HeadHunter, готовит отклики и показывает controlled automation flow.



Реальный результат автоотклика: система сама нашла вакансию, сопоставила её с профилем и откликнулась — на видео звонок работодателя после этого отклика. Самая неожиданная вакансия, которую нашёл бот: предлагать людям контракт на военную службу. https://t.me/annsflow/184

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Browser Runtime

- Перевёл проект с API-first на browser-first архитектуру через Playwright и Chromium.
- Реализовал управление реальной HH-сессией через persistent Chromium profile.
- Добавил health-check авторизации, bootstrap-login и single-writer lock для browser profile.
- Построил browser transport adapter вместо прямого HH API transport.
- Сохранил доменную логику поиска, анализа, выбора резюме, генерации письма и логирования.

Apply Flow & Questionnaires

- Реализовал ApplyFlow как state machine: открыть страницу, выбрать резюме, вставить письмо, подготовить черновик, отправить отклик.
- Добавил классификацию страниц отклика: simple, questionnaire, already applied, fixed resume, unknown UI, auth-required.
- Сделал controlled auto-submit для простых вакансий без анкеты.
- Построил questionnaire engine для textarea, input, radio, checkbox, select и mixed-вопросов.
- Добавил LLM-ответы на анкеты с honest fallback logic и final approval.

Safety & Human-in-the-loop

- Финальный submit невозможен без approval_token=APPROVED.
- Режим auto_with_handoff отправляет простые вакансии автоматически, а сложные передаёт в Telegram approval.
- Telegram-кнопки Approve / Reject / Skip дают ручной контроль чувствительных действий.
- Safety layer включает throttling, circuit breakers, quarantine mode и cooldown.
- Система различает SKIP / NEEDS_HUMAN и реальные инфраструктурные ошибки.

State, Retry & Observability

- Антидубли между запусками через state-файлы и SQLite audit log.
- Retry queue сохраняет незавершённые вакансии между сессиями.
- Trace, screenshots и HTML artifacts сохраняются при ошибках browser runtime.
- Dashboard pending approvals показывает ручные кейсы в браузере.
- Ежедневная статистика и Telegram-отчёты строятся из SQLite application log.

LLM & Prompts

- LLM-классификация вакансий: APPLY / SKIP / MANUAL_REVIEW / DELAY.
- Генерация персонализированного cover letter под конкретный JD.
- Structured output для классификации и questionnaire drafts.
- Prompt injection guard через явные content boundaries для текста вакансии.
- Salary parsing hardening и fallback-логика для вопросов о зарплате.

Scheduler & Regression

- Auto runner работает как планировщик запусков через macOS LaunchAgents.
- Operational profiles: обычный 24x7 профиль и burst profile.
- Квоты по отправкам за запуск и за час защищают runtime от перегруза.
- Regression corpus покрывает apply-page variants и scheduler behavior.
- Preflight-проверки валидируют runtime, session health, safety state и active defaults.

▼ Показать полный technical scope

- Persistent Chromium profile, health-check авторизации, bootstrap-login и profile lock.
- Browser-based поиск вакансий, извлечение карточек и full vacancy details через DOM-локаторы.
- Domain/transport contracts: VacancyCard, VacancyDetails, ApplyDraft, ApplyResult, StatusUpdate, InboxUpdate.
- ApplyPage abstraction, PageVariant taxonomy и fallback-сценарии для нестабильного HH UI.
- Controlled auto-submit, Telegram approval workflow, pending approvals resume и dedup approval requests.
- Questionnaire engine: text, textarea, email, phone, number, radio, checkbox, select, mixed option + textarea.
- Retry queue, startup resume, attempt limits и escalation в needs_human.
- Safety layer: throttling, circuit breakers, quarantine mode, cooldown, bounded navigation и wall-clock guard.
- Artifacts: trace, screenshots, HTML, run summary, daily stats и Telegram reports.
- Browser pollers для статусов откликов, переписок и negotiation messages.
- Operational profiles, runtime_profile_ctl, scheduler guard, quotas и burst mode.
- OpenAI SDK v2, structured output, prompt injection guard, salary parsing hardening и fallback policies.
- Atomic JSON-state writes, corrupt JSON backup, stale temp cleanup и synthetic hygiene.
- Regression levels: preflight, draft smoke, approval/notify smoke, controlled submit regression.

- Production invariants: не обходить CAPTCHA, не делать submit без approval token, не мутировать applied IDs до подтверждённого submit.

Python Playwright Chromium OpenAI API SQLite Telegram Bot API pytest LaunchAgents JSON state

Portfolio repo: https://github.com/workflowarsenynn/job-application-automation-portfolio

---

03 · LLM + INTAKE

Avito Legal Bot для юридической компании

Система первичной обработки юридических заявок из Avito Messenger: сценарный диалог, LLM-анализ, FSM-контроль, SQLite-состояние и запись структурированного лида в Google Sheets.

ЗАДАЧА

Клиенты приходили из Avito сырыми сообщениями: данные были неполными, юристы тратили время на первичные вопросы, бот не должен был давать консультации или обещать результат.

РЕШЕНИЕ

Построен гибрид FSM + LLM: сценарий контролирует порядок и безопасность диалога, LLM анализирует сообщения, извлекает факты и формулирует живые ответы внутри guardrails. Результат сохраняется в SQLite и Google Sheets как структурированная карточка клиента.

КАК РАБОТАЕТ

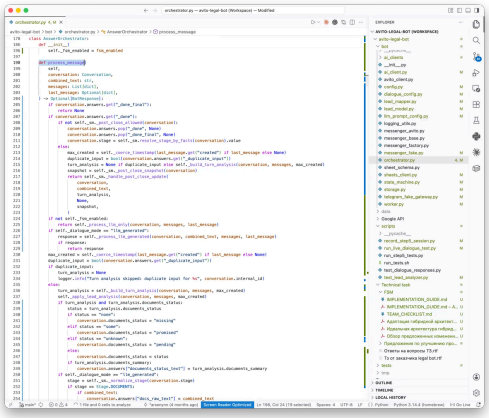
Avito message → batching/dedup → Turn Analysis через LLM → обновление фактов клиента → Stage Router/FSM → безопасный ответ → Lead Analysis → Google Sheets CRM → replay/tests для проверки поведения.

РОЛЬ

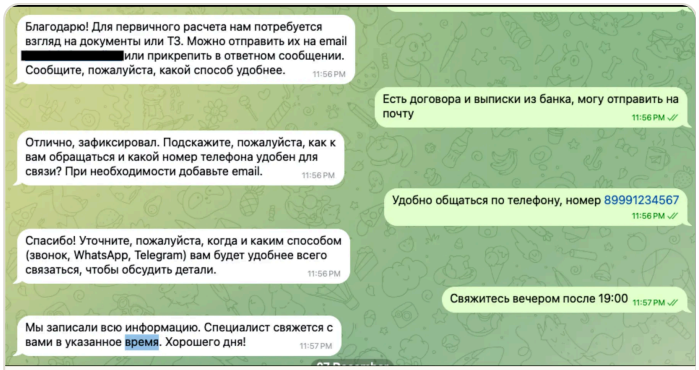
Архитектура диалога, FSM, LLM-orchestration, guardrails, storage, Avito/Sheets интеграции, тесты и документация.

РЕЗУЛЬТАТЫ

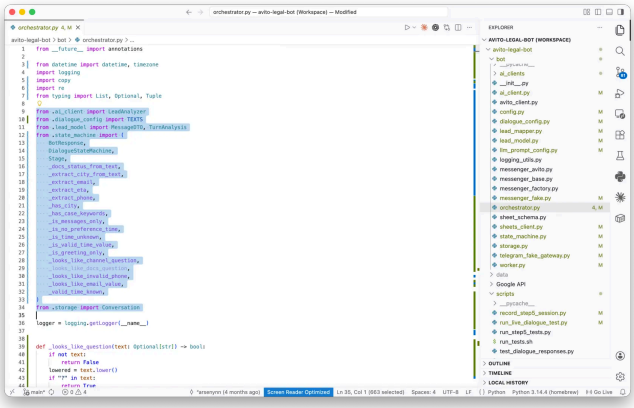
- Структурированная карточка лида вместо сырой переписки
- FSM не даёт боту пропускать обязательные этапы
- LLM отвечает живо, но не нарушает юридические guardrails
- SQLite сохраняет состояние и историю диалога
- Google Sheets получает один диалог как одну CRM-строку
- Replay, BDD и live-smoke тесты проверяют поведение



Runtime-flow обработки сообщения: состояние диалога, LLM-анализ, дедупликация, обновление фактов и переход по FSM.



Telegram fake gateway позволяет тестировать Avito Legal Bot в реальном времени без боевого Avito-чата.



orchestrator.py связывает LLM-анализ, FSM, storage и обработку фактов в единый pipeline.

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Dialogue Flow

- Полный сценарий первичного юридического диалога: ситуация, город, документы, канал отправки, контакт и время связи.
- FSM жёстко контролирует порядок этапов и не даёт боту перескакивать через важные шаги.
- Stage Router определяет актуальную стадию по уже собранным фактам.
- Бот не спрашивает повторно телефон, документы, город или канал, если клиент уже сообщил данные.
- Есть отдельная ветка для нецелевых обращений: контакт сохраняется для передачи партнёрам.

LLM Orchestration

- FSM отвечает за правила и безопасность, LLM отвечает за живую формулировку сообщений.
- Turn Analysis извлекает город, документы, канал, контакт, время связи и намерение клиента.
- Lead Analysis формирует структурированную карточку лида для Google Sheets.
- Строгая JSON-схема делает LLM-ответы предсказуемыми.
- Fallback-политика регенерирует ответ или откатывается на безопасный шаблон.

Legal Guardrails

- Боту запрещено давать полноценные юридические консультации и прогнозы исхода дела.
- Ответы остаются краткими первичными пояснениями и возвращают клиента к сбору данных.
- Документы уточняются только после понимания ситуации и города.
- Телефон в приоритете, при отказе запрашивается альтернативный контакт.
- Служебные контакты компании не записываются как контакт клиента.

Storage & CRM

- SQLite хранит разговоры, сообщения, стадии, ответы, документы и технические флаги.
- Google Sheets работает как CRM-таблица юридических лидов.
- Append/update логика по internal ID не создаёт дубли строк.
- Отложенная запись в Sheets защищает диалог от сбоев таблицы.
- Сохраняются контакт, город, услуга, документы, канал отправки, время связи, комментарий и статус.

Integrations & Runtime

- Avito API получает чаты, сообщения и отправляет ответы клиентам.
- Telegram gateway имитирует клиентов и показывает ответы в реальном времени.
- Единый messenger-интерфейс позволяет менять Avito/fake-режим без изменения бизнес-логики.
- .env управляет Avito, Sheets, OpenAI, SQLite, Telegram fake-mode и режимами диалога.
- Worker polling-loop обрабатывает новые сообщения, отвечает и синхронизирует таблицу.

Tests & Diagnostics

- BDD-сценарии в YAML проверяют диалоги как пользовательские истории.
- Replay-механизм воспроизводит живые LLM-ответы без повторных вызовов OpenAI.
- Fake AI и heuristic AI дают детерминированные тесты.
- Acceptance-тесты проверяют соответствие исходным бизнес-требованиям.
- LLM-as-judge оценивает естественность и соответствие правилам.

▼ Показать полный technical scope

- Avito Messenger intake, FSM-сценарий, LLM-orchestrated и LLM-generated режимы.
- Turn Analysis, Lead Analysis, Stage Router, strict JSON schema и post-processing LLM-текста.
- Дедупликация входящих сообщений, дедупликация исходящих ответов и batching быстрых сообщений клиента.
- Google Sheets append/update по internal ID, маппинг в фиксированную схему колонок и delayed write при сбоях.
- FSM tests, JSON parsing tests, Orchestrator tests, dedup tests, restart resilience tests.
- Production-профиль thin: LLM пишет естественно, FSM контролирует этапы, запреты и обязательные условия.
- Обработка ранних данных, повторных вопросов, soft-must времени связи и post-close update.
- SQLite storage для разговоров, сообщений, стадий, ответов, статусов документов и технических флагов.
- Avito token refresh, fake Telegram mode, messenger factory, .env config и polling worker.
- Live-smoke, replay JSONL, test artifacts, stage trace summary, technical guide и implementation guide.

Python | Avito API | OpenAI API | SQLite | Google Sheets API | Telegram fake gateway | FSM | BDD/YAML tests

Portfolio repo: https://github.com/workflowarsenynn/legal-assistant-automation-portfolio

---

04 · PARSER + LEAD GEN

EventRadar — парсер Telegram-каналов

Отдельный сервис для автоматического поиска горячих заявок на техническое обеспечение мероприятий в Telegram-чатах. Система слушает профессиональные сообщества через Telethon, фильтрует шум, классифицирует лиды и отправляет чистые заявки в backend основного бота.

ЗАДАЧА

Менеджеры вручную мониторили десятки Telegram-чатов, пропускали горячие заявки, ловили дубли и тратили внимание на вакансии, рекламу и нерелевантный шум.

РЕШЕНИЕ

EventRadar собирает сообщения через Telethon, нормализует их в RawMessage, применяет pre-filter, основной фильтр и LLM-классификатор, формирует LeadCandidate и отправляет лиды в backend бота через защищённый webhook.

КАК РАБОТАЕТ

Telegram-чаты → Telethon listener → RawMessage → soft pre-filter → lead classifier / LLM → LeadCandidate JSON → backend /eventradar/lead → Telegram-уведомление продажам.

РОЛЬ

Архитектура сервиса, Telegram client, pre-filter, LLM-классификация, модели данных, backend contract, конфиги и диагностика.

РЕЗУЛЬТАТЫ

- Автоматизирован мониторинг профессиональных event-сообществ
- LLM + эвристики отделяют заявки от шума
- Лиды доставляются в тот же поток, что Avito и site chat
- Bearer-token защищает внутренний endpoint
- Score и reason объясняют, почему сообщение стало лидом
- Supervisor-cycle восстанавливает сервис после временных сбоев

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Telegram Source Layer

- Сбор сообщений из выбранных Telegram-чатов через Telethon под рабочим аккаунтом.
- Прослушивание нескольких event-чатов в реальном времени или с минимальной задержкой.
- YAML-конфиг для добавления новых источников без переписывания кода.
- Тестовый Telegram-чат для безопасной проверки логики.
- Скрипт для получения chat_id и списка Telegram-диалогов.

Filtering Pipeline

- RawMessage сохраняет текст, источник, автора, чат, ссылку, reply, forward, media и сервисный статус.
- Soft pre-filter отсекает очевидный шум до дорогой обработки.
- Фильтруются сервисные сообщения, пустые медиа, бессодержательные reply/forward и короткие символьные ответы.
- Pre-filter безопасен: он не режет потенциальные заявки агрессивно.
- Режимы off, soft и зарезервированный hard вынесены в конфиг.

Lead Classification

- Основной фильтр определяет, похоже ли сообщение на запрос по техобеспечению мероприятия.
- Распознаются звук, свет, DJ с аппаратурой, экраны, проекторы, сцена, видео и другое оборудование.
- Категории оборудования: sound, dj_with_gear, lights, led_screen, plasma, projector, stage, video_production, other.
- LLM-классификатор принимает смысловое решение, эвристики страхуют при ошибках.
- Извлекаются город, дата, бюджет, тип события, reason, score и категории.

Backend Contract

- Лиды отправляются в backend основного Telegram-бота через HTTP POST.
- Авторизация между EventRadar и backend идёт через Bearer token.
- JSON-формат LeadCandidate унифицирует передачу лида.
- Endpoint /eventradar/lead валидирует токен, принимает JSON и пересылает уведомление.
- EventRadar подключён как отдельный источник, не ломая Avito и web-chat архитектуру.

Runtime & Recovery

- .env, .env.local и YAML управляют Telegram API, backend URL, token, фильтрами и LLM.
- Telethon session-файл позволяет работать без повторной авторизации.
- Supervisor-cycle перезапускает сборщик при сетевых сбоях, ошибках LLM или недоступности backend.
- Exponential backoff увеличивает задержку до 5 минут при повторяющихся ошибках.
- KeyboardInterrupt корректно останавливает сервис как daemon или Launch Agent.

Product Logic

- Система помогает находить заявки раньше конкурентов сразу после появления в чате.
- Шум превращается в поток потенциальных лидов с объяснимой классификацией.
- Структура уведомлений готова к маршрутизации по оборудованию, городу, бюджету и типу мероприятия.
- Score и reason позволяют анализировать ложные срабатывания и улучшать prompt.
- Проект является частью единой системы входящих обращений из Avito, сайта и Telegram-сообществ.

▼ Показать полный technical scope

- Telethon client, saved session, YAML chat config, test chat и chat_id discovery script.
- Pre-filter modes: off, soft, reserved hard, minimum text length и DEBUG reasons.
- OpenAI-compatible LLM config: API base, model, key, timeout и safe disable при неполной конфигурации.
- HTTP POST backend integration, Bearer token, /eventradar/lead contract и separate EventRadar group routing.
- Supervisor loop, exponential backoff, graceful shutdown, daemon/Launch Agent readiness.
- RawMessage и LeadCandidate модели для унифицированного lifecycle сообщения.
- Lead classifier с категориями оборудования и video_production routing.
- Hybrid LLM + heuristics, permissive mode и legacy stricter heuristic mode.
- Centralized .env/.env.local/YAML, validation, safe defaults и warnings по неизвестным режимам.
- README, release checklist, compileall, unit-testable pre-filter/LLM/filter boundaries.

Python | Telethon | OpenAI-compatible LLM | YAML config | HTTP webhook | Bearer token

Telegram Bot backend

GitHub: в подготовке

05 · RAG + KNOWLEDGE SYSTEMS

Obsidian RAG Core

Backend-first read-only RAG-система поверх Obsidian Vault. Проект индексирует Markdown, Canvas-графы, вложения и OCR-текст, строит retrieval-индекс в SQLite + Qdrant и отвечает через API с sources, evidence snippets и retrieval metadata.

ЗАДАЧА

Обычный поиск по заметкам плохо работает с большой базой знаний: он находит похожие куски текста, но не понимает контекст проекта, связи между идеями, структуру Canvas и границы области поиска.

РЕШЕНИЕ

Система читает Obsidian Vault в read-only режиме, строит собственный индекс, превращает Canvas в semantic graph, поддерживает scope-aware retrieval и отдаёт ответы через FastAPI с источниками, evidence snippets и retrieval diagnostics.

КАК РАБОТАЕТ

Vault discovery → Markdown/Canvas/OCR ingestion → SQLite metadata → Qdrant vector store → scope-aware retrieval → Canvas graph expansion → LLM answer → sources/evidence/retrieval metadata.

РОЛЬ

Архитектура RAG-системы, ingestion pipeline, Canvas IR parser, metadata model, retrieval policies, model adapters, API layer и roadmap retrieval quality / latency.

РЕЗУЛЬТАТЫ

- Read-only backend: vault остаётся source of truth
- Canvas-aware retrieval превращает .canvas в semantic graph
- Scope-aware search по всем проектам, одному проекту или выбранным scopes
- Evidence-first answers с sources и snippets
- OpenAI и локальные Ollama adapters
- Debug search, retrieval timing и semantic traces

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Vault Ingestion

- Read-only backend читает Obsidian Vault без изменения исходных заметок.
- Project discovery определяет folder note, _MOC.md, Canvas, Markdown, attachments и legacy media.
- Markdown разбирается на retrieval-документы с metadata и chunking.
- Attachment ingestion поддерживает PDF text layer и optional OCR.
- Incremental indexing обновляет только изменившиеся источники.

Canvas Understanding

- .canvas файлы превращаются в Canvas IR, а не отправляются в модель как сырой JSON.
- Извлекаются nodes, edges, groups, labels, colors, directions, file references и subpaths.
- Canvas semantic layer переводит визуальные связи в retrieval-сигналы.
- Graph expansion учитывает соседние элементы и ограничивает глубину traversal.
- Semantic trace объясняет, почему Canvas был прочитан именно так.

Retrieval Core

- SQLite хранит metadata layer, источники, hash-значения и lookup-данные.
- Qdrant используется как vector store для dense retrieval.
- Scope-aware retrieval не подмешивает unrelated projects без явного разрешения.
- Retrieval profiles explore и fact_evidence используют разные policies.
- Evidence-first answer assembly возвращает sources и snippets.

Model Backends

- OpenAI adapters для embeddings и generation.
- Ollama adapters для локальных embeddings и generation.
- Model abstraction layer позволяет менять backend без переписывания retrieval core.
- Fallback paths деградируют при недоступности внешних backend'ов.
- Локальный запуск подтверждён на Mac Studio с Ollama, Qdrant и FastAPI.

API & Debuggability

- FastAPI endpoints для full reindex, project reindex, changed-only indexing, query и debug search.
- /query возвращает answer + sources.
- /search/debug показывает scope-aware hits без генерации.
- Retrieval metadata показывает backend, model, timings и параметры retrieval.
- Indexing summary logs помогают контролировать процесс индексации.

Roadmap & Evaluation

- Зафиксирован roadmap перехода с 256-dim embeddings на 512/1024-dim embeddings.
- Запланирован настоящий BM25 вместо упрощённого sparse scoring.
- Scope и metadata filtering должны перейти в Qdrant payload filters.
- Планируется async /query и streaming generation.
- Regression retrieval set и benchmarks нужны для оценки retrieval quality.

▼ Показать полный technical scope

- Read-only RAG backend, Projects indexing, project boundary model и strict scope-aware retrieval.
- Canvas IR parser: nodes, file nodes, link nodes, groups, edges, edge labels, directions, subpaths, colors.
- Attachment ingestion: PDF text layer, OCR via tesseract, OCR retrieval-documents.
- Dense retrieval, initial hybrid fusion, profile-aware ranking, per-source cap и source deduplication.
- OpenAI/Ollama adapters, local model stack, fallback paths и vendor-agnostic model abstraction.
- Debug endpoints, retrieval timings, semantic traces, regression retrieval set и evaluation layer.
- Markdown ingestion: folder notes, _MOC.md, frontmatter, wiki-links, headings и project-relative paths.
- Canvas semantic layer: precedence rules, alias resolution, semantic trace, graph expansion и partial reindex.
- SQLite metadata layer, Qdrant vector store, stable document IDs, qdrant-client query_points compatibility.
- Sensitivity-aware metadata, max_sensitivity, exclude_sensitive, access_context и clearance-ready filters.
- Full indexing, incremental indexing, symlink-aware ingestion, canonical source identity и hash-based detection.
- Roadmap: stronger embeddings, BM25, Qdrant payload filters, async flow, streaming generation, query embedding cache.

Python FastAPI SQLite Qdrant Obsidian Canvas Markdown Ollama OpenAI API Docker OCR

Private / in progress

06 · MACOS AUTOMATION + LUA

Mac Automation Stack — клавиатурные слои и управление рабочим местом

Персональная система автоматизации macOS на связке Karabiner-Elements + Hammerspoon. Karabiner превращает физические клавиши в чистые сигналы: Hyper, tap/hold-слои, виртуальные F-клавиши, правила по раскладке. Hammerspoon на Lua превращает эти сигналы в действия: запуск приложений, оконный менеджер, медиа-сценарии, авто-раскладку мониторов, HTTP-пульт для Shortcuts и MIDI-контроллер для OBS.

ЗАДАЧА

Готовые утилиты дают ровно тот набор функций, который придумал автор: как только нужны слои, контекст, надёжные фолбэки и своя логика для окон и медиа, упираешься в ограничения интерфейса. Системные хоткеи зависят от фокуса и анимаций, многомониторный сетап разъезжается после переподключения дисплеев, а медиа в каждом приложении управляется по-своему.

РЕШЕНИЕ

Система разделена на два уровня. Karabiner-Elements отвечает за «железо ввода»: модульные JSON-правила задают Hyper на Caps Lock, слои Space (hold = F18) и backtick (hold = F17) и точечные правила для RU-раскладки. Hammerspoon отвечает за поведение: Lua-конфиг слушает эти сигналы и выполняет сценарии с проверками состояния, задержками и фолбэками — от запуска приложений и тайлинга окон до HTTP-сервера, displayplacer и управления OBS по WebSocket.

КАК РАБОТАЕТ

Физическая клавиша → JSON-правило Karabiner (tap/hold, Hyper, F17/F18, input source) → чистый сигнал → hotkey или modal layer в Hammerspoon → Lua-сценарий (проверка состояния → фокус → задержка → команда → fallback) → приложение, окно, медиа, монитор или OBS. Параллельно: Shortcuts → HTTP endpoint на Mac, MIDI-нота → действие в command mode.

РОЛЬ

Архитектура слоёв ввода, JSON-правила Karabiner, Lua-конфиг Hammerspoon: оконный менеджер, медиа-сценарии, HTTP-сервер с авторизацией и watchdog, авто-раскладка мониторов, MIDI → OBS WebSocket, синхронизация конфига и серия разборов в Telegram-канале.

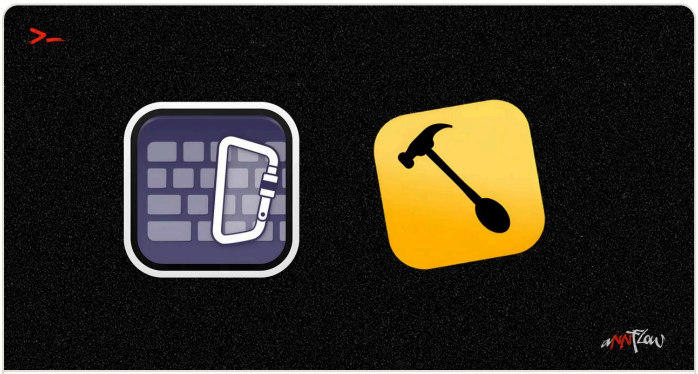
РЕЗУЛЬТАТЫ

- Запуск и фокус приложений одним сочетанием Hyper + клавиша — без Dock и поиска
- Три слоя ввода — Hyper, Space-hold и backtick-hold — не ломают обычную печать
- Окна раскладываются мгновенно и детерминированно, с возвратом «как было»
- Медиа-команды адресно уходят в Spotify, YouTube, Twitch, Music, Podcasts и Books
- Удалённый пульт: HTTP API для Shortcuts с токеном, rate limit и watchdog
- MIDI-клавиатура работает как аппаратная панель для сцен OBS

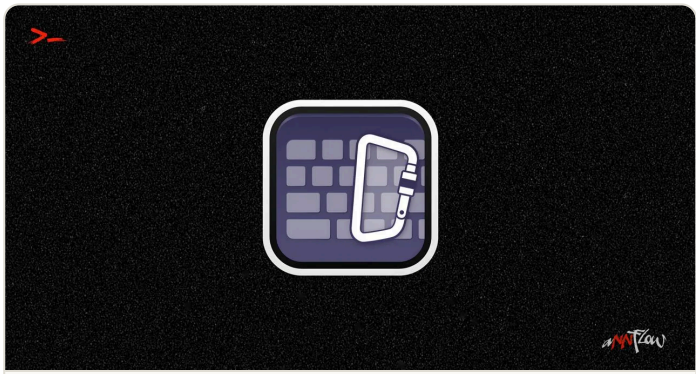
ПОДРОБНЫЙ РАЗБОР

Серия постов #macOSAutomation: от «железа» к логике — слои Karabiner, сценарии Hammerspoon, окна, медиа, HTTP-пульт и MIDI → OBS.

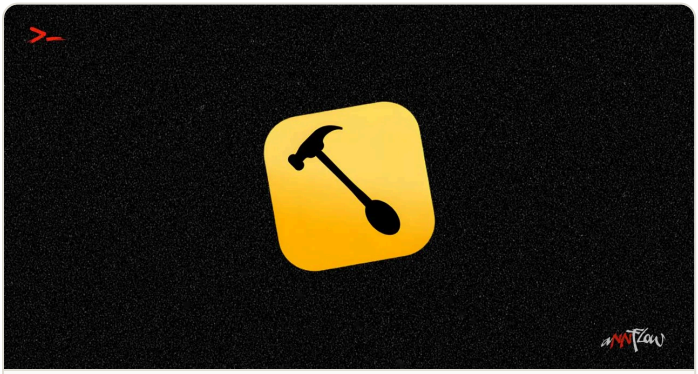
https://t.me/annsflow/74



Связка Karabiner + Hammerspoon: Karabiner отвечает за ввод, Hammerspoon — за логику и сценарии.



Karabiner-Elements: модульные JSON-правила превращают физические клавиши в чистые сигналы — Hyper, F17, F18.



Hammerspoon: Lua-конфиг слушает сигналы и выполняет сценарии — окна, медиа, мониторы, HTTP-пульт и MIDI.

ФАЙЛЫ ПРОЕКТА

- Caps Lock = Hyper — https://portfolio.arsenynn.com/downloads/mac-automation/caps_lock_to_hyper.json
- Space: tap = пробел, hold = F18 — https://portfolio.arsenynn.com/downloads/mac-automation/space_hold_f18.json
- Backtick: tap = `, hold = F17 — https://portfolio.arsenynn.com/downloads/mac-automation/grave_hold_f17.json
- RU: Option + , / . → запятая и точка — https://portfolio.arsenynn.com/downloads/mac-automation/ru_option_punct.json

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Karabiner Input Layer

- Karabiner-Elements как «компилятор ввода»: физические клавиши превращаются в предсказуемые сигналы.
- Модульные Complex Modifications: один JSON-файл = одна идея, правила включаются, тестируются и откатываются по отдельности.
- Caps Lock = Hyper (cmd+ctrl+option+shift): суперклавиша под мизинцем и чистый слой хоткеев без конфликтов.
- Space: tap = пробел, hold = виртуальная F18; порог удержания подобран так, чтобы слой не включался при печати.
- Backtick: tap = символ, hold = виртуальная F17 — вход в Leader Key-иерархию.
- RU-раскладка: Option + , и Option + . печатают запятую и точку только в русском input source.

Hammerspoon Core & Layers

- Lua-конфиг init.lua как единый движок логики поверх сигналов Karabiner.
- Hyper + клавиша: запуск и фокус конкретных приложений через hs.application.open и launchOrFocus.
- Space-layer на F18: модальный режим с медиа-контролем и запуском дополнительных приложений.
- Backtick-layer на F17: Leader Key-подслои — F открывает рабочие папки в Finder, G — проекты в ChatGPT.
- Конфиг хранится как код и синхронизируется через iCloud Drive симлинком — одинаковое поведение на всех Mac.

Window Manager

- Слой Ctrl + Option: A и D — левая и правая половина, S — центр и другие мышечно-запоминаемые команды.
- hs.window.animationDuration = 0: окна меняют геометрию мгновенно, без зависимости от анимаций.
- moveToUnit для тайлинга в долях экрана и setFrame для точных зон — например, правая колонка 25%.
- Toggle «разложить / вернуть»: исходный frame сохраняется в windowStates, раскладка обратима.
- Сетка окон текущего приложения с восстановлением позиций повторным запуском.
- Перенос между мониторами по топологии west/east/north/south и шаговая подстройка по 100 px.

Media Scenarios

- Не универсальная пауза, а сценарий под каждое приложение: активация → пауза на фокус → команда → fallback.
- Spotify: AppleScript playpause напрямую, без эмуляции клавиш и зависимости от активного окна.
- YouTube, Twitch и Safari: фокус → выход из Space-layer → задержка 50 мс → клавиша k.
- Apple Music и Podcasts: фокус → задержка → space как play/pause.
- Apple Books: UI scripting через System Events с учётом RU/EN-названий меню и fallback на space.
- Если приложение не запущено — alert вместо команды «в никуда».

HTTP Control Server

- Hammerspoon поднимает локальный HTTP-сервер на фиксированном порту — Mac Studio работает как хаб.
- Авторизация через header X-Auth-Token вместо Basic Auth для стабильной интеграции с Shortcuts.
- Rate limit по минимальному интервалу между вызовами защищает от дублей и гонок.
- Health-check /ping и watchdog перезапускают сервер, если он перестал отвечать.
- Endpoints: медиа-команды по приложениям и системные действия, включая pmset displaysleepnow.
- Клиенты — Shortcuts на iPhone и других Mac: кнопка → HTTP-запрос → действие на Mac.

Displays & Hardware Control

- displayplacer и отслеживание подключения дисплеев по persistent ID, включая iPad/Sidecar.
- Авто-применение лэйаутов «с iPad» и «без iPad» после короткой паузы на стабилизацию macOS.
- После отключения Sidecar окна с крайнего экрана возвращаются на primary display.
- MIDI-клавиатура как аппаратная панель: карта smkActions[note] с label и fn для каждой ноты.
- Command mode включается sustain-педалью (CC64); фильтры velocity, диапазон 25 клавиш и debounce.
- Управление OBS по WebSocket: кеш списка сцен с коротким TTL и SetCurrentProgramScene по индексу.

▼ Показать полный technical scope

- Karabiner-Elements Complex Modifications: модульные JSON-файлы в ~/.config/karabiner/assets/complex_modifications.
- Tap/hold-правила: Space → F18, backtick → F17, подбор to_if_held_down_threshold_milliseconds.
- Hammerspoon init.lua: hs.hotkey.bind, modal layers, Leader Key-подслои для Finder и ChatGPT.
- Window management через hs.window: moveToUnit, setFrame, moveToScreen, windowStates, нулевая анимация.
- Медиа-сценарии: AppleScript для Spotify, hs.eventtap.keyStroke с hs.timer.doAfter, UI scripting через System Events.
- Hyper-модификатор на Caps Lock (cmd+ctrl+option+shift) на уровне Karabiner, а не сторонней утилиты.
- Условия по input source: правила пунктуации работают только в русской раскладке и не трогают EN.
- Синхронизация конфига через iCloud Drive симлинком на стандартный путь ~/.hammerspoon/.
- Сетка окон приложения, обратимые раскладки, перенос по топологии экранов и шаговые сдвиги.
- hs.httpserver: фиксированный порт, X-Auth-Token, rate limit, /ping health-check и watchdog с автоперезапуском.

- Интеграция с Apple Shortcuts на iPhone и других Mac как клиентами HTTP API.
- hs.midi: обработка noteOn, velocity, диапазона нот, CC64 command mode, слоёв действий и debounce.
- displayplacer, persistent display ID, скрипты лэйаутов и возврат окон на primary display после отключения Sidecar.
- OBS WebSocket: кеш сцен с TTL, obsSetSceneByIndex и SetCurrentProgramScene.

Karabiner-Elements Hammerspoon Lua JSON AppleScript HTTP server OBS WebSocket MIDI displayplacer Apple Shortcuts iCloud

Personal setup · ежедневное использование

07 · SPEECH-TO-TEXT + LOCAL AI

VoxBase — голос, видео и подкасты в Markdown-память

Локальная система транскрибации, которая превращает записи с iPhone, TikTok, YouTube и Apple Podcasts в Markdown с metadata. Silero VAD находит речь, ffmpeg готовит сигнал, whisper-cli с моделью large-v3 распознаёт текст на Metal GPU, а launchd-worker обрабатывает очередь автономно — без облачного транскрибатора, оплаты за минуты и передачи разговоров сторонним сервисам.

ЗАДАЧА

После переговоров и встреч остаётся много нюансов, которые невозможно удержать в голове, а аудиофайл на телефоне сам по себе ничего не решает. Встроенные решения iPhone ограничены по качеству, контролю и формату результата, облачные сервисы требуют ручной загрузки и подписки, а важные видео и подкасты теряются в ленте.

РЕШЕНИЕ

Построен локальный конвейер: запись с iPhone через iCloud попадает на Mac, worker проверяет готовность файла, Silero VAD строит карту речи, речевые участки собираются в chunks примерно по 60 секунд, ffmpeg очищает сигнал, whisper-cli large-v3 распознаёт текст. Результат сохраняется как Markdown с metadata о происхождении. ClipMind добавляет второй вход: ссылки из TikTok, YouTube и Apple Podcasts идут по самому дешёвому пути к тексту и попадают в Todoist.

КАК РАБОТАЕТ

Запись на iPhone → iCloud → launchd-worker раз в минуту проверяет готовность → рабочая копия → Silero VAD → chunks ~60 с речи → ffmpeg (mono, 16 kHz, denoise, loudnorm) → whisper-cli large-v3 на Metal GPU → Markdown + metadata в iCloud → архив оригинала. Ссылка через Share Sheet → ClipMind → captions / transcript / RSS / локальный Whisper → Markdown → Todoist.

РОЛЬ

Архитектура конвейера, VAD- и chunking-логика, сигнальный тракт ffmpeg, локальный Whisper runtime, launchd-worker и защита от повторной обработки, разделение iCloud-артефактов и локального state, Voice Reference, ClipMind с маршрутизацией по источникам и интеграцией с Todoist.

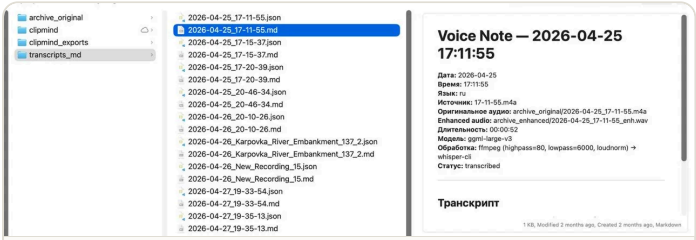
РЕЗУЛЬТАТЫ

- Транскрипт готов через несколько минут после записи — без ручных шагов
- VAD пропускает тишину: 62 минуты аудио → 42:50 речи, 30,9% не пришлось транскрибировать
- 26-минутная запись: 131 речевой сегмент собран в 4 chunks без потери контекста
- Полностью локально: Whisper large-v3 на Metal GPU, без оплаты за минуты и лимитов
- Оригиналы не изменяются, каждый транскрипт хранит metadata о происхождении
- ClipMind: TikTok, YouTube и Apple Podcasts превращаются в Markdown и задачу в Todoist

ПОДРОБНЫЙ РАЗБОР

Серия постов #VoxBase: почему голосовые заметки умирают, VAD и chunks, сигнальный тракт, локальный worker, Voice Reference и ClipMind.

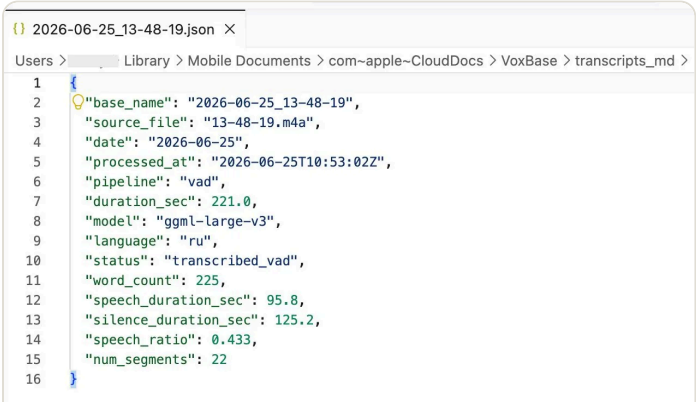
https://t.me/annsflow/155



transcripts_md: рядом с каждой записью появляются Markdown-транскрипт и JSON-metadata — дата, язык, модель, обработка и статус.

```
"Processing 2026-07-10_18-39-50|via VAD|Running Silero|segments,|Slicing into ch
unks|merged .* chunks|Transcribing chunks|type.*large v3|using device Metal|GPU
name|Markdown →|Done ✓|Worker finished" \
"$HOME/Library/Application Support/VoxBase/logs/2026-07-10.log"
[2026-07-10T18:41:02] INFO: ── Processing 2026-07-10_18-39-50 ──
[2026-07-10T18:41:02] INFO: [VAD] Processing 00:00:18 via VAD pipeline
[2026-07-10T18:41:02] INFO: [VAD] Running Silero VAD...
OK: 3 segments, 16.8s speech
[2026-07-10T18:41:03] INFO: [VAD] speech_ratio=0.913 (3 segments, 00:00:16 speec
h)
[2026-07-10T18:41:03] INFO: [VAD] Slicing into chunks...
  merged 3 segments → 1 chunks (target=60.0s speech each)
[2026-07-10T18:41:04] INFO: [VAD] Transcribing chunks...
whisper_model_load: type          = 5 (large v3)
whisper_default_buffer_type: using device Metal (Apple M2 Max)
ggml_metal_init: GPU name:   Apple M2 Max
[2026-07-10T18:41:08] INFO: [VAD] Markdown → transcripts_md/2026-07-10_18-39-50.
md
[2026-07-10T18:41:08] INFO: [VAD] Done ✓ 2026-07-10_18-39-50 (speech: 00:00:16,
silence skipped: 8.7%)
[2026-07-10T18:41:08] INFO: Worker finished. Processed 1 file(s).
```

Один рабочий цикл: Silero VAD находит речь, chunks уходят в Whisper large-v3 на Apple M2 Max (Metal), Markdown создаётся автоматически.



Metadata транскрипта: из 221 секунды записи речью оказались 95,8 секунды — speech_ratio, сегменты, модель и pipeline сохраняются вместе с текстом.

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Capture & Intake

- Запись голоса на iPhone через Just Press Record, iCloud Drive работает как курьер между устройствами.
- Проверка готовности: файл уже не .icloud, размер перестал меняться, запись ещё не обработана.
- Lock от параллельного запуска и защита от повторной обработки одного и того же файла.
- Работа только с рабочей копией: исходный .m4a не режется, не чистится и не перезаписывается.
- Оригинал архивируется после обработки и помечается как processed.

VAD & Chunking

- Silero VAD строит карту речи: где человек говорит, а где пауза, шум или тишина.
- Мелкие речевые сегменты склеиваются в chunks примерно по 60 секунд речи.
- Короткие фрагменты по 1–2 секунды не отправляются в Whisper — так модель не теряет контекст и не галлюцинирует.
- Тишина не транскрибируется: на часовой записи пропускается почти треть файла.
- Транскрипт сохраняет исходные таймкоды записи.

Signal Chain

- ffmpeg-цепочка перед распознаванием: highpass 80 Hz и lowpass 6 kHz срезают гул и шипение.
- afftdn подавляет постоянный фоновый шум, acompressor 4:1 выравнивает тихие и громкие фразы.
- loudnorm нормализует итоговую громкость, сигнал приводится к mono 16 kHz WAV.
- Основной режим очищает запись, не съедая окончания слов и тихую речь.
- Отдельный Noisy preset для сложных условий записи.

Local Runtime & Storage

- whisper-cli с моделью large-v3 на Metal GPU — внешний API для транскрибации не нужен.
- launchd запускает worker раз в минуту; очередь переживает отсутствие интернета и остановку worker.
- В iCloud — только артефакты: Markdown, metadata и постоянные копии оригиналов.
- Локально — chunks, временное аудио, логи, state, ошибки, locks и cache, чтобы не раздувать CloudKit.
- Ночной агент удаляет временные enhanced-аудио; оригиналы не удаляются никогда.

Voice Reference

- Эталон голоса из нескольких образцов: iPhone, микрофон Mac, разные условия, по 1–3 минуты.
- pyannote разделяет запись по говорящим, SpeechBrain строит voice embeddings.
- Каждый голос сравнивается с эталоном: наибольшее совпадение получает метку Me, остальные — Other.
- Score совпадения сохраняется в metadata: уверенно, Me с предупреждением или low confidence.
- Принцип: не скрывать неопределённость, а хранить её вместе с результатом.

ClipMind Media Intake

- Ссылка с iPhone через Share Sheet попадает в одну очередь, источник определяется автоматически.
- TikTok: captions с проверкой языка, объёма текста, lyrics и мусорных автосубтитров, иначе локальный Whisper.
- YouTube: готовый transcript без скачивания видео; при отсутствии субтитров — metadata и понятный статус ошибки.
- Apple Podcasts: iTunes Lookup → RSS → transcript; тяжёлый fallback на аудио запускается только явно.
- Задача в Todoist создаётся сразу и обновляется по мере обработки — ссылка не теряется даже при ошибке.
- Временные видео, аудио и chunks очищаются, в iCloud остаются только лёгкие финальные артефакты.

▼ Показать полный technical scope

- Конвейер iPhone → iCloud → Mac worker → VAD → chunks → Whisper → Markdown + metadata.
- Single-instance worker с lock, рабочие копии и архив оригиналов без модификации.
- ffmpeg: highpass, lowpass, afftdn, acompressor, loudnorm, mono 16 kHz; основной и Noisy preset.
- Markdown-транскрипт с заголовком: дата, время, язык, источник, длительность, модель, обработка, статус.
- Готовность файлов iCloud: проверка .icloud-плейсхолдеров, стабилизации размера и processed-реестра.
- Silero VAD, merge сегментов в chunks с target ~60 с речи и сохранением исходных таймкодов.
- whisper-cli + ggml-large-v3 на Metal GPU (Apple Silicon), язык ru.
- JSON-metadata: duration, speech/silence duration, speech_ratio, num_segments, word_count, pipeline, status.

- launchd StartInterval 60, восстановление очереди после офлайна iPhone и остановки worker.
- Ночная очистка временных enhanced-аудио, оригиналы хранятся постоянно.
- ClipMind: роутинг TikTok / YouTube / Apple Podcasts, captions quality checks, iTunes Lookup и RSS.
- Разделение хранения: iCloud для артефактов, локальный диск для chunks, логов, state, locks и cache.
- Voice Reference: pyannote diarization, SpeechBrain embeddings, similarity score и метки Me / Other.
- Todoist-интеграция: задача создаётся при постановке в очередь, обновляется без дублей, failure artifacts при ошибке.

Python · Whisper large-v3 · whisper-cli · Silero VAD · ffmpeg · Metal GPU · launchd · iCloud Drive · pyannote · SpeechBrain · Todoist API · Markdown

Private · ежедневное использование

---

08 · STREAMING + PYTHON

Without Streaming — Telegram-канал как радиостанция

Интернет-радио, которое превращает музыкальный Telegram-канал в непрерывный эфир. Новый пост в канале автоматически становится частью программы, Python решает, что прозвучит дальше, Liquidsoap формирует сигнал, а HLS через Caddy доставляет один общий live-эфир всем слушателям — с визуалами, PWA и аналитикой прослушиваний.

ЗАДАЧА

Личная музыкальная медиатека жила в Telegram-канале как лента постов: чтобы включить музыку в компании или в машине, нужно было листать канал и запускать треки по одному. Нужен был эфир в один клик — без отдельной админки, ручной сборки плейлистов и с поведением, которое не ломается на iOS, в WebView и при сворачивании браузера.

РЕШЕНИЕ

Проект прошёл путь от персонального плеера к радиостанции. Первая версия на FastAPI + Telethon индексировала канал, собирала playlist.json с тегами стиля и энергии и стримила аудио из Telegram chunked без сохранения на диск. Вторая версия стала настоящим радио: медиатека зеркалируется локально, Python программирует эфир, Liquidsoap играет и делает переходы, сигнал кодируется в AAC-LC и раздаётся через HLS и Caddy — все слушатели подключаются к одному и тому же моменту эфира.

КАК РАБОТАЕТ

Пост с треком в Telegram → индексатор читает аудио, визуалы и теги → локальная медиатека → Python выбирает следующий трек → Liquidsoap воспроизводит и сводит переходы → AAC-LC → HLS-сегменты → Caddy → браузер или PWA с одним постоянным <audio> подключается к текущему моменту эфира, визуалы MP4 вращаются независимо.

РОЛЬ

Идея и продукт, индексатор Telegram-канала, теговая модель и алгоритм ранжирования эфира, FastAPI-стриминг, браузерный плеер и PWA, переход на архитектуру Liquidsoap + HLS + Caddy, локальная медиатека и operational layer.

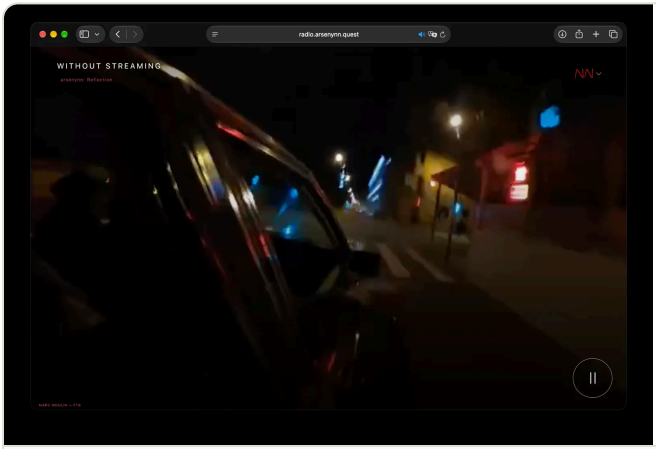
РЕЗУЛЬТАТЫ

- Эфир в один клик: открыл ссылку — и слушаешь тот же момент, что и все
- Telegram работает как CMS: новый пост сам попадает в эфир, без админки
- Эфир не зависит от доступности Telegram — медиатека зеркалируется локально
- Play после паузы возвращает в текущий эфир, а не к старой позиции трека
- Устойчивый клиент: фоновый режим, PWA и gesture gate для iOS и WebView
- Operational layer: посетители, реальные HLS-слушатели, сессии за день и peak

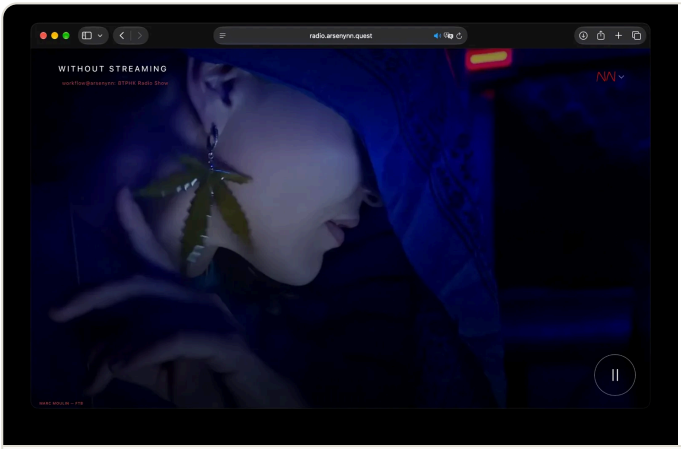
ПОДРОБНЫЙ РАЗБОР

Серия постов #WithoutStreaming: радио из Telegram в один клик, ручки настроения, конвейер индексатор → плейлист → стрим, плеер и PWA, переход к общему эфиру.

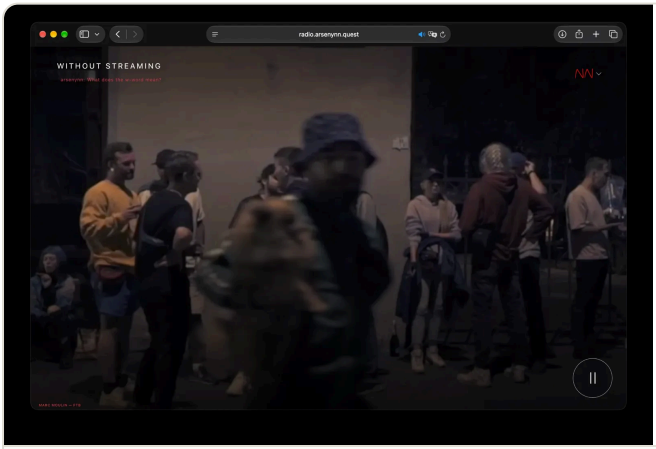
https://t.me/annsflow/93



Live-эфир в браузере: полноэкранный визуал, источник клипа под логотипом, текущий трек внизу и одна кнопка Play/Pause.



Визуалы — короткие вырезки из проектов @arsenynn и workflow@arsenynn — вращаются независимо от музыкальной программы.



Минимальный интерфейс: слушатель подключается к текущему моменту эфира, без плейлистов и лишних кнопок.

ЖИВОЙ ПРОЕКТ

Without Streaming — включить эфир в браузере или на телефоне.

https://radio.arsenynn.quest

ВИДЕО PROOF

Запись экрана iPhone: Without Streaming в браузере — один общий эфир, полноэкранные визуалы и управление одной кнопкой.

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Telegram as CMS

- Telegram-канал — единственная точка публикации: новый пост с треком автоматически становится частью эфира.
- Индексатор пробегает канал, находит аудио и GIF/MP4-визуалы и читает теги.
- Теги стиля и энергии: #J / #H / #JxH и #drift / #drive / #dd, смешанные треки работают как мосты.
- Каталог собирается в playlist.json без отдельной админки и медиатеки.
- Telegram из runtime-зависимости превратился в CMS и источник происхождения материала.

Mood Engine (v1)

- Две ручки как на пульте: Jazz ⇄ Hip-hop и Drifting ⇄ Driving.
- /playlist?mix=&energy= пересобирает порядок треков под положение ползунков.
- Положение ручек превращается в коэффициенты bias, веса трека берутся из тегов, итог — score.
- Сначала профильные треки и мосты, затем подмешиваются остальные, чтобы эфир не зацикливался.
- Auto Flow и Auto Energy двигают ручки плавно и не ломают текущий трек.

Telegram Streaming (v1)

- FastAPI + Telethon: /audio/{id} читает трек из Telegram порциями и сразу отдаёт в браузер.
- StreamingResponse с audio/mpeg: воспроизведение начинается до полной загрузки файла.
- Аудио не сохраняется на диск — минимум инфраструктуры и минимум следов.
- /gif/{id} отдаёт визуалы с кэшем на диске, потому что они короткие и повторяются.
- Визуалы как телеканал: каждый повторяется три раза, стартовый набор перемешан.

Radio Station Core (v2)

- Один общий эфир для всех: слушатель подключается к тому, что звучит прямо сейчас.
- Python решает, что должно прозвучать дальше; Liquidsoap — фактическая точка эфира.
- Liquidsoap непрерывно воспроизводит треки и делает переходы между ними.
- Сигнал кодируется в AAC-LC и нарезается в HLS-сегменты для live-доставки.
- Caddy раздаёт HLS, сайт и статические файлы; медиатека зеркалируется локально.

Player, PWA & Platforms

- v1: плеер как state machine idle → buffering → playing, кроссфейд и префетч следующего трека.
- Режим «Слушать в фоне»: облегчённый профиль на простом <audio> для iOS и WebView.
- PWA для предсказуемого запуска и более долгой жизни сессии.
- Gesture gate для Telegram Mini App и Safari WebView: визуалы ждут первого жеста вместо «молча не играет».
- v2: один постоянный <audio> на HLS, Play после паузы возвращает в текущий эфир.

Operations & Roadmap

- Operational layer: сколько людей на сайте и сколько реально слушает HLS.
- Дневное количество listening sessions и peak слушателей.
- Визуалы подготовлены локально и раздаются как статические MP4 независимо от музыкальной программы.
- Следующий этап: самовосстановление после падений процессов, потери сети и reboot.
- Замер пропускной способности домашнего сервера, затем серверный Programming Engine для общего эфира.

▼ **Показать полный technical scope**

- Telegram-канал как CMS, индексатор аудио и GIF/MP4-визуалов, теговая модель стиля и энергии.
- Смешивание профильных треков, мостов и остального каталога; Auto Flow / Auto Energy.
- Кэш визуалов на диске и ротация «телеканала» с перемешанным стартовым набором.
- PWA и gesture gate для Telegram Mini App и Safari WebView с осмысленной деградацией autoplay.
- Один общий live-эфир, подключение к текущему моменту и возврат в эфир после паузы.
- Метрики: посетители сайта, реальные HLS-слушатели, listening sessions за день и peak.
- playlist.json и /playlist?mix=&energy= с ранжированием по bias × веса тегов = score.
- FastAPI + Telethon: StreamingResponse, chunked-отдача аудио без записи на диск, Cache-Control no-store.
- Браузерный плеер v1: state machine, кроссфейд, префетч, фон-режим на <audio>.
- v2: Telegram → локальная медиатека → Python → Liquidsoap → AAC-LC → HLS → Caddy.
- Независимая ротация статических MP4-визуалов у каждого слушателя.
- Roadmap: auto-recovery после сбоев процессов, сети и reboot, нагрузочный замер домашнего сервера, Programming Engine.

Python Liquidsoap HLS AAC-LC Caddy FastAPI Telethon Telegram JavaScript PWA Web Audio

Live · radio.arsenynn.quest

09 · LLM + MEMORY

ann — LLM-бот с памятью и рециркуляцией контента

Telegram-бот ann для сообщества «Бессмысленность и скоротечность»: LLM-собеседник с собственной памятью в SQLite, накопительными профилями участников, смысловым поиском по медиатеке и философией, загружаемой в runtime как правила речи. Бот отвечает на ситуацию разговора, а не на последнюю реплику, и возвращает старые материалы в поток, когда для них появляется место.

ЗАДАЧА

В соцсетях материал тонет в потоке через день после публикации, а обычный чат-бот забывает собеседника после каждого окна диалога и отвечает безлико. Нужна была среда, где разговор не обнуляется, контент продолжает жить годами, а бот говорит в узнаваемом голосе проекта — без цитирования правил и без хаоса из дублей и повторов.

РЕШЕНИЕ

Построена многослойная система поверх Telegram: SQLite хранит сообщения со структурой ответов, медиа, профили и их evidence; перед ответом бот собирает контекст из базы и профиль человека; медиатека проиндексирована с embeddings и возвращается в разговор по смысловой близости; философия проекта хранится в JSON, загружается в runtime и превращается в правила генерации. Тяжёлые операции вынесены в фон, а база и код защищают систему от дублей и повторных действий.

КАК РАБОТАЕТ

Сообщение в чате → запись в SQLite (текст, автор, reply, entities, медиа) → решение, нужно ли отвечать → контекст из истории чата с лимитами → блок профиля участника → system prompt с философией ann → LLM-ответ → поиск релевантного контента по embeddings → возврат материала в чат. В фоне: накопление реплик → пересборка профиля, ротация контента и Stories с cooldown.

РОЛЬ

Идея и концепция среды, архитектура слоёв, схема памяти SQLite, сборка контекста и профилей, embeddings-поиск по медиатеке, загрузка философии в runtime и prompt-правила, медиа-слой и Stories, защита от дублей, фоновые очереди и серия разборов в Telegram-канале.

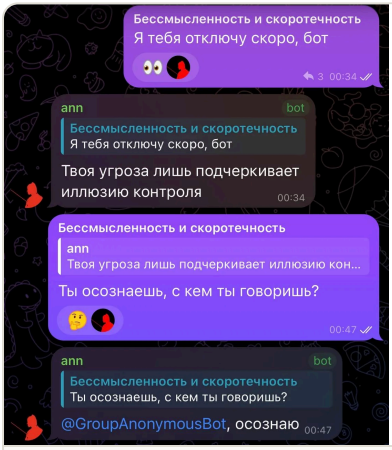
РЕЗУЛЬТАТЫ

- Разговор не обнуляется: бот помнит историю, авторов и структуру ответов
- Ответы адресны: профиль участника накапливается и уточняется по мере общения
- Медиатека рециркулируется: материал возвращается в разговор по смысловой близости
- Узнаваемый голос: философия загружается в runtime и управляет стилем речи
- Нет дублей и лишних действий: UNIQUE, INSERT OR IGNORE и cooldown
- Тяжёлая логика в фоне — диалог остаётся быстрым, сбои не ломают поток

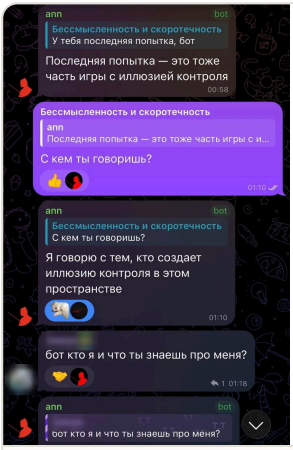
ПОДРОБНЫЙ РАЗБОР

Серия постов #annBot: идея среды вместо книги, слои системы, память и профили, логика ответов, рециркуляция контента, философия в коде и защита от хаоса.

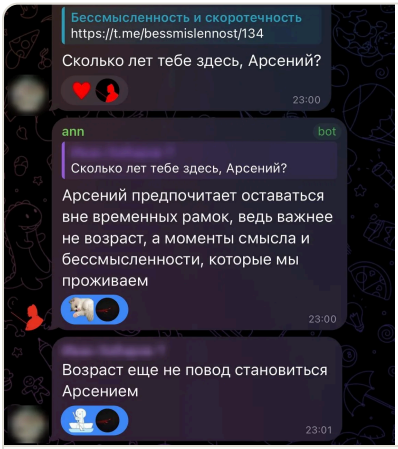
https://t.me/annsflow/107



Голос проекта в действии: ann отвечает не шаблоном ассистента, а в своей философии — даже на попытку «отключить» бота.



Ответ на ситуацию разговора: бот учитывает цепочку реплик и отвечает разным участникам в одном голосе.



Разговор вокруг материала: вопрос под репостом из канала, ответ бота в логике «смысла и бессмысленности».

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Memory Layer

- SQLite как собственная память бота: сообщения, пользователи, профили, evidence, медиа и служебное состояние.
- ann_messages хранит чат, сообщение, автора, время, текст, reply и entities — структуру и форму разговора.
- Нормализация типов и безопасные дефолты при записи: событие сохраняется, даже если часть полей пуста.
- Разделение по chat_id, чтобы разные диалоги не смешивались.
- SQLite в режиме WAL: запись не блокирует систему целиком.

Context & Response

- Сначала решение, нужно ли отвечать, и только потом генерация.
- Контекст собирается из базы с лимитами по числу сообщений и символам, чтобы prompt не раздувался.
- Отдельный блок профиля подбирает тон и факты под последнее сообщение участника.
- Финальный пакет: system prompt с характером ann, profile block и контекст разговора.
- Бот отвечает на ситуацию разговора, а не на одну последнюю реплику.

Profiles & Evidence

- Профиль — обновляемая модель присутствия человека, а не анкета, заполненная один раз.
- Счётчик новых реплик и порог PROFILE_UPDATE_EVERY: профиль не пересобирается на каждое сообщение.
- Пересборка вынесена в фоновую очередь queue_profile_update.
- ann_profiles хранит текущее описание, ann_profile_evidence — основания, из которых оно выросло.
- Профили делают ответы адресными, но не цитируются дословно.

Content Recirculation

- Отдельная база контента с описаниями материалов и embeddings для смыслового поиска.
- Ленивая однократная загрузка индекса content.json в память.
- pick_relevant_content ищет top-k материалов, близких к текущей теме разговора.
- Найденный материал отправляется обратно в чат — старые вещи получают новую точку входа.
- Медиатека работает как живая среда, а не мёртвый архив.

Philosophy as Runtime

- Философия проекта хранится в ann_philosophy.json и загружается в runtime-состояние.
- Fallback на кэш: при ошибке чтения основной файл не теряется.
- Структуры identity, postulates, style, channels и primary_source управляют поведением.
- Prompt-правила: не цитировать внутреннее знание и профиль дословно, отвечать по смыслу последнего сообщения.
- Лексические опоры стиля превращают мировоззрение в механизм отбора фраз и интонации.

Media, Stories & Reliability

- ann_message_media фиксирует voice, audio, photo, video_note и sticker с file_id, подписью и источником.
- UNIQUE(chat_id, msg_id, media_type) и INSERT OR IGNORE отсекают повторные события Telegram.
- ann_story_state управляет публикацией Stories: источник, использованный файл и cooldown_until.
- Тяжёлые операции — очереди и отложенные обновления вне диалога.
- Паузы, fallback-сценарии и восстановление состояния при сбоях внешних сервисов.

▼ Показать полный technical scope

- Telegram Bot: сообщения, команды, reply-цепочки, медиа и события участников чата.
- Сохранение reply_to_msg_id и entities_json для памяти о структуре и форме разговора.
- Profile prompt block с бюджетом токенов для обычных ответов и упоминаний.
- Накопительное профилирование с порогом обновления и фоновой очередью пересборки.
- ann_philosophy.json с fallback-кэшем: identity, postulates, style, channels, primary_source.
- Медиа-слой: voice, audio, photo, video_note, sticker с дедупликацией по file_unique_id и сообщению.
- SQLite (WAL): ann_messages, ann_message_media, ann_profiles, ann_profile_evidence, ann_profile_counters, ann_story_state.
- Сборка контекста из базы с лимитами ANN_CONTEXT_MSG_LIMIT и ANN_CONTEXT_MAX_CHARS.
- Многослойный prompt: system prompt, profile block, контекст разговора.
- Content index в JSON с embeddings, top-k смысловой поиск и отправка материала в чат.
- Prompt-правила против дословного цитирования внутреннего знания и профиля.
- Stories: состояние публикации, источник материала и cooldown_until.

- Защита от хаоса: UNIQUE, INSERT OR IGNORE, cooldown, фоновые очереди, fallback и восстановление состояния.

Python Telegram Bot API OpenAI API SQLite (WAL) Embeddings Prompt engineering JSON config Telegram Stories

Live · Telegram-сообщество

---

10 · REAL-TIME AI + KNOWLEDGE

LiveContext — живой разговор, связанный с памятью проекта

Система реального времени, которая слушает звонок двумя независимыми аудиопотоками, локально транскрибирует речь, отслеживает состояние разговора и прямо во время звонка передаёт в LLM нужный контекст проекта из Obsidian. Модель уже знает номер заказа, претензии, сроки и историю обращений — пока разговор ещё идёт.

ЗАДАЧА

Сохранить разговор после звонка — уже решённая задача (VoxBase), но контекст часто нужен прямо во время разговора: собеседник спрашивает о прошлой неделе, появляется новое возражение или нужен конкретный факт проекта. Информация уже лежит в Obsidian, документах и прошлых разговорах, но искать её вручную, пока человек ждёт ответа, почти бессмысленно.

РЕШЕНИЕ

LiveContext разделяет голоса ещё до транскрибации: микрофон идёт отдельным потоком YOU, звук приложения звонка — через виртуальное аудиоустройство потоком REMOTE. Каждый поток проходит свою обработку и очередь, затем локальную транскрибацию. Система ведёт состояние разговора, выбирает только нужный контекст конкретного проекта из Obsidian и передаёт его в модель вместе с текущей репликой — без поиска LLM по всей базе знаний и без автоматической перезаписи памяти.

КАК РАБОТАЕТ

Микрофон → YOU, Телемост или браузер → Loopback «LiveContext Remote» → REMOTE → отдельная обработка, определение речи и очередь для каждого потока → локальная транскрибация → состояние разговора → выбор релевантного контекста проекта в Obsidian → LLM получает реплику и контекст → подсказка во время звонка.

РОЛЬ

Архитектура real-time конвейера, разделение аудиопотоков YOU / REMOTE и виртуальное устройство, обработка и очереди для каждого потока, локальная транскрибация, модель состояния разговора, выбор контекста из Obsidian, правила работы LLM с памятью и серия разборов в Telegram-канале.

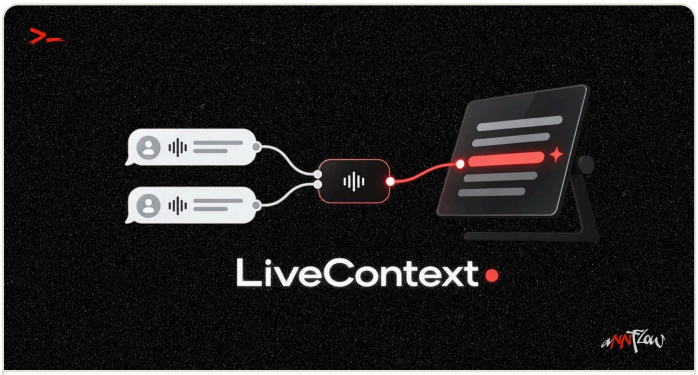
РЕЗУЛЬТАТЫ

- Нужные факты проекта появляются во время разговора, а не после него
- Кто говорит — известно из источника звука, а не угадывается по тексту
- Модель знает номер заказа, претензии, сроки и историю обращений по ходу звонка
- Obsidian работает как оперативная память, а не только как архив
- LLM получает только контекст текущего проекта, а не всю базу знаний
- Локальная транскрибация двух потоков без внешнего API

ПОДРОБНЫЙ РАЗБОР

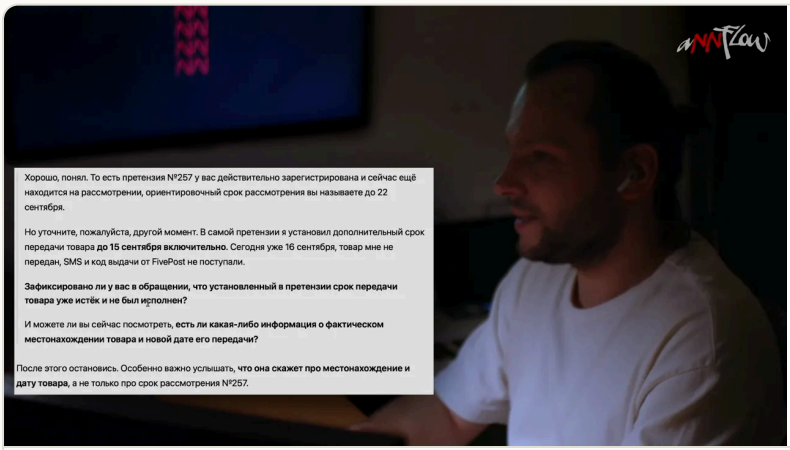
Серия постов #LiveContext: зачем памяти работать во время разговора, два аудиопотока YOU / REMOTE и LiveContext в реальном звонке.

https://t.me/annsflow/186



LiveContext: два голосовых потока сходятся в одну систему, которая выводит нужный контекст прямо во время разговора.

ВИДЕО PROOF



Реальный звонок по претензии о недоставленном заказе: LiveContext анализирует разговор в реальном времени и передаёт его в модель вместе с контекстом проекта из Obsidian — модель подсказывает формулировки и следующие вопросы, пока разговор ещё идёт. https://t.me/annsflow/189

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Dual Audio Capture

- Голоса разделяются ещё до транскрибации — без программного угадывания спикеров.
- YOU: физический микрофон, AirPods или USB-микрофон.
- REMOTE: звук Телемоста или браузера через виртуальное устройство «LiveContext Remote» в Loopback.
- Компьютер получает две независимые аудиодорожки вместо одной смешанной.
- Источник звука сразу задаёт роль реплики: «клиент сказал» и «я сказал» не путаются.

Stream Processing & Transcription

- Для каждого потока — своя обработка, свои параметры определения речи и своя очередь.
- Локальная транскрибация без внешнего API.
- Два непрерывных потока обрабатываются так, чтобы не положить весь компьютер.
- Whisper — только одна из частей: основная сложность в подготовке потоков и их синхронной обработке.
- Опыт локального конвейера VoxBase перенесён в режим реального времени.

Conversation State

- Разговор рассматривается не как transcript, а как текущее состояние.
- Роли YOU / REMOTE — часть состояния: одинаковый текст означает разное в зависимости от того, кто говорит.
- Система отслеживает смену темы, новые возражения и вопросы о прошлых договорённостях.
- Цепочка: разговор → транскрибация → состояние → контекст проекта → нужная информация.

Context Selection

- LiveContext определяет, какой контекст нужен именно сейчас, по текущей реплике и состоянию.
- LLM не ищет по всей базе знаний — она получает ограниченный контекст конкретного проекта.
- Одна реплика превращается в адресный запрос к нужным заметкам Obsidian.
- Меньше шума в prompt — точнее подсказка в момент, когда собеседник ждёт ответа.

Obsidian as Working Memory

- Obsidian превращён из архива в оперативную память проекта.
- В контекст попадают факты проекта, документы, правила и прошлые разговоры.
- Транскрипты VoxBase и заметки проекта работают как единая память.
- Память не переписывает себя автоматически: изменения в базе знаний остаются под контролем человека.

Live Assist

- Разговор и актуальный контекст проекта передаются в модель по ходу звонка.
- Модель знает номер заказа, претензии, сроки и историю обращений.
- Подсказки формулировок и следующих вопросов появляются, пока разговор ещё идёт.
- Проверено на реальном звонке по претензии о недоставленном заказе.

▼ Показать полный technical scope

- Real-time конвейер: два аудиопотока → транскрибация → состояние разговора → контекст проекта → LLM.
- Виртуальное аудиоустройство «LiveContext Remote» в Loopback для Телемоста и браузера.
- Локальная транскрибация без внешнего API, контроль нагрузки на компьютер.
- Выбор релевантного контекста из Obsidian по текущей реплике, без поиска LLM по всей базе.
- Память без автоматической самоперезаписи: изменения базы знаний — под контролем человека.
- Раздельный захват: физический микрофон (YOU) и звук приложения звонка (REMOTE).
- Независимая обработка, параметры определения речи и очереди для каждого потока.
- Модель состояния разговора с ролями YOU / REMOTE.
- Obsidian как оперативная память: заметки проекта, документы, правила и транскрипты прошлых разговоров.
- Связка с VoxBase (память после разговора) и Obsidian RAG Core (поиск по базе знаний).

Python · Whisper (local) · VAD · Loopback · Obsidian · LLM · macOS audio · Markdown

Private · в разработке

11 · MCP + KNOWLEDGE SYSTEMS

Obsidian MCP — доступ LLM к проектной памяти

MCP-сервер, через который языковые модели работают с реальным Obsidian-vault на Mac: ищут по проектам, открывают implementation_plan.md и decisions_log.md, собирают контекст и правят заметки по прямому запросу. Память остаётся в локальных файлах и продолжает быть source of truth, а внешняя LLM достаёт её через Cloudflare, VPS и reverse SSH — без копирования vault в облако.

ЗАДАЧА

Проектов, решений и заметок стало слишком много, чтобы держать контекст в голове: после нескольких недель на другой задаче его приходилось восстанавливать заново. Вся нужная информация уже лежала в Obsidian, но языковая модель её не видела — каждый раз нужно было вручную искать и копировать файлы в диалог, а симметричная синхронизация Obsidian и Todoist порождала вопрос, какая из систем главная.

РЕШЕНИЕ

Сначала разделены память и выполнение: Obsidian хранит архитектуру, планы и историю решений, Todoist — только то, что нужно сделать сейчас, по правилу «один факт = один владелец». Затем построен MCP-сервер как интерфейс между LLM и Obsidian: модель получает набор инструментов (структура проекта, поиск по vault, чтение и правка заметки) и сама добирается до нужных документов по протоколу «сначала поиск, потом чтение». Для доступа внешних моделей сервер на Mac опубликован через Cloudflare и VPS reverse SSH-туннелем, а vault никогда не покидает компьютер.

КАК РАБОТАЕТ

Вопрос о проекте → MCP-сессия (initialize → tools/list) → LLM выбирает инструмент → поиск в области проекта одним широким запросом → MCP возвращает подходящие файлы и фрагменты → модель читает только нужные документы → ответ. Транспорт: LLM / MCP-клиент → Cloudflare → VPS → reverse SSH → MCP server на Mac → Obsidian Local REST API → vault.

РОЛЬ

Архитектура разделения памяти и выполнения, правило «один факт = один владелец», MCP-сервер и набор инструментов для Obsidian, search-first протокол доступа, транспорт Cloudflare → VPS → reverse SSH → Mac, автозапуск и восстановление, переход к Obsidian Knowledge Agent и серия разборов в Telegram-канале.

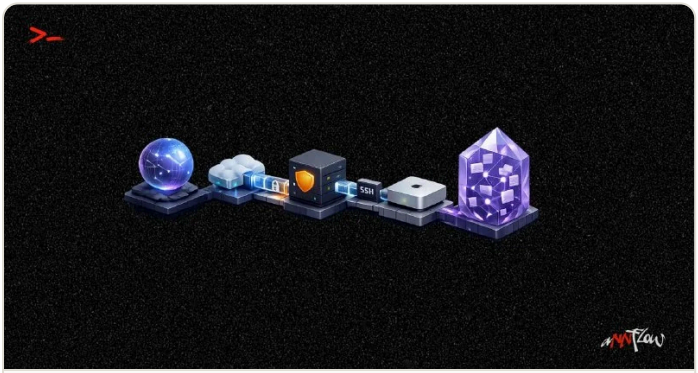
РЕЗУЛЬТАТЫ

- LLM восстанавливает состояние проекта из реальных файлов — без ручного копирования в чат
- Vault остаётся локальным source of truth: в облаке нет второй копии памяти
- Search-first протокол: модель читает только нужные документы, а не весь vault
- Один факт = один владелец: Obsidian хранит память, Todoist управляет выполнением
- Удалённая модель работает с Obsidian на Mac через Cloudflare и reverse SSH
- Туннель и сервер самовосстанавливаются: LaunchAgent и autossh

ПОДРОБНЫЙ РАЗБОР

Серия постов #ObsidianMCP: разделение памяти и выполнения, доступ LLM к Obsidian, search-first поиск, путь запроса до локального Mac и следующий этап — Knowledge Agent.

https://t.me/annsflow/171



Путь запроса: внешняя LLM → Cloudflare → VPS → reverse SSH → MCP-сервер на Mac → Obsidian vault, который остаётся локальным.

ЧТО ВНУТРИ СИСТЕМЫ

Ключевые модули и инженерные слои проекта

Memory vs Execution

- Obsidian — проектная память: архитектура, планы реализации, технические паспорта, история решений и Canvas.
- Todoist — execution-слой: только то, что нужно сделать сейчас, а не вторая копия памяти.
- Отказ от симметричной синхронизации, где непонятно, какая система главная.
- Правило «один факт = один владелец»: решения принадлежат памяти, действия — выполнению.
- Состояние проекта живёт в файлах: implementation_plan.md и decisions_log.md.

MCP Tools for Obsidian

- MCP-сервер работает как формальный интерфейс между языковой моделью и Obsidian.
- Инструменты: структура проекта, поиск заметки, поиск по vault, чтение файла.
- Правка обычной заметки — только по прямому запросу.
- Весь Obsidian не загружается в модель — LLM получает инструменты, а не содержимое.
- Сервер обращается к vault через Obsidian Local REST API.

Search-first Protocol

- Сначала задаётся область поиска — папка конкретного проекта.
- Один широкий запрос с OR-условиями вместо перебора предполагаемых имён файлов.
- Из результатов читаются только наиболее релевантные документы.
- Не читать весь проект заранее и не продолжать запросы после 429.
- Протокол появился после реального rate limit Obsidian MCP и экономит контекст модели.

MCP Session Flow

- Клиент создаёт MCP-сессию через initialize и получает mcp-session-id.
- tools/list отдаёт модели список доступных инструментов.
- tools/call вызывает конкретный инструмент с параметрами.
- Модель сама решает, какую «дверь» в память открыть, и идёт дальше по необходимости.
- Цепочка: вопрос → поиск → найденные файлы → чтение → сбор контекста → ответ.

Remote Transport

- Публичный MCP endpoint принимает Cloudflare и передаёт трафик на VPS через cloudflared.
- На VPS нет MCP-сервера: порт создаётся reverse SSH-туннелем, который поднимает сам Mac.
- Туннель доступен только внутри VPS и ведёт на локальный MCP-порт Mac.
- VPS — только транспорт: не хранит vault, не индексирует заметки, не копирует память.
- MCP server и туннель запускаются LaunchAgent, autossh восстанавливает соединение после обрывов.

Next: Knowledge Agent

- MCP хорошо решает задачу доступа, но модель сама собирает контекст.
- Obsidian Knowledge Agent добавляет собственный поиск и индекс, понимание структуры проектов, связей и Canvas.
- Правила источников и границ проекта определяют, какой контекст передавать модели.
- Чтение, рассуждение и изменение разделены: правки — только через контролируемый механизм с правилами безопасности.
- MCP остаётся одним из способов подключения: LLM → Knowledge Agent → проектная память.

▼ Показать полный technical scope

- Архитектура «память отдельно от выполнения»: Obsidian (source of truth) и Todoist (execution).
- Правило «один факт = один владелец», состояние проекта в implementation_plan.md и decisions_log.md.
- MCP-сервер для Obsidian: инструменты структуры проекта, поиска по vault, чтения и правки заметок.
- Интеграция с Obsidian Local REST API, работа с реальным vault без копий.
- Search-first протокол: scoped searchInPath, широкий OR-запрос, чтение только релевантных файлов.
- Обработка rate limit: без перебора имён файлов и без повторных запросов после 429.
- MCP-сессия: initialize, mcp-session-id, tools/list, tools/call.
- Транспорт: MCP-клиент → Cloudflare → cloudflared на VPS → reverse SSH → MCP server на Mac.
- Туннель доступен только внутри VPS; VPS не хранит и не индексирует данные.
- LaunchAgent для MCP-сервера и туннеля, autossh для автоматического восстановления.

- Roadmap: Obsidian Knowledge Agent — собственный индекс, структура проектов и Canvas, правила источников, контролируемые изменения.

MCP · Obsidian · Obsidian Local REST API · LLM · Cloudflare · VPS · reverse SSH · autossh · LaunchAgent · Todoist · Markdown

Private · ежедневное использование

КОНТАКТ

Открыт к AI / automation задачам и техническим ролям

Telegram: @arsenynnn · Email: arsenynickolaevich@yandex.ru · GitHub: workflowarsenynn · Telegram-канал: https://t.me/annsflow